

Experiment: Tic-Tac-Toe Game using Python (Minimax Algorithm)

Aim

To develop a Python program for the **Tic-Tac-Toe** game using the **Minimax algorithm** in Artificial Intelligence.

Objective

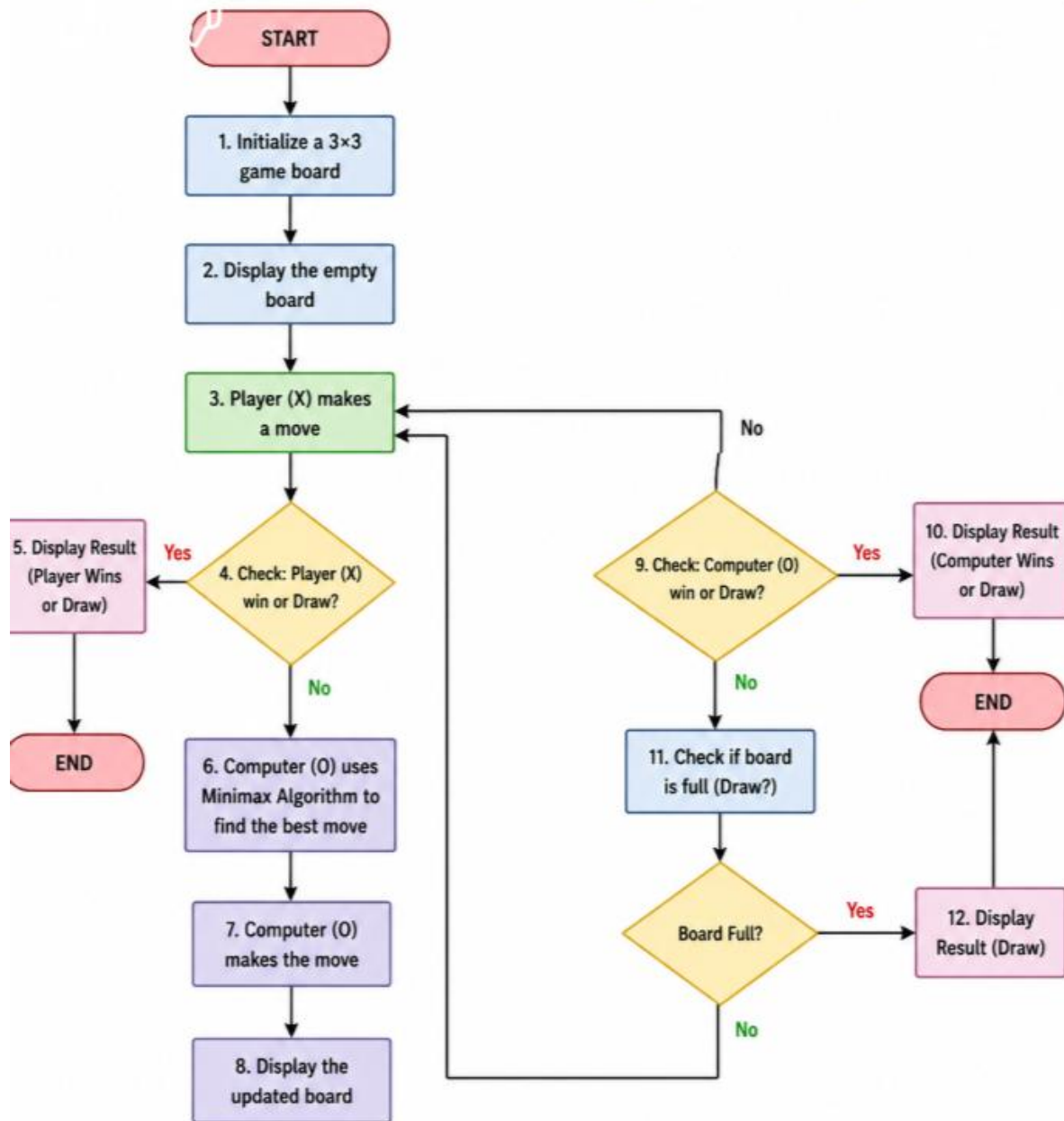
To implement an AI-based Tic-Tac-Toe game where the computer makes optimal moves using the Minimax algorithm, ensuring the best possible outcome.

Algorithm

1. Start.
2. Initialize a 3×3 game board.
3. Allow the human player (X) to make a move.
4. Check whether the game has been won or drawn.
5. If the game is not over, use the **Minimax algorithm** to determine the best move for the computer (O).
6. Place the computer's move on the board.
7. Repeat Steps 3–6 until a player wins or the game ends in a draw.
8. Display the game result.
9. Stop.

FLOWCHART

Flowchart: Tic-tac-toe Game using Minimax Algorithm



Code:

```
import math
```

```
board = [' ' for _ in range(9)]
```

```
# Display board
```

```
def print_board():
```

```
print()
for i in range(0, 9, 3):
    print(board[i], "|", board[i+1], "|", board[i+2])
    if i < 6:
        print("--+---+--")
print()
```

Check winner

```
def check_winner(player):
    wins = [
        [0,1,2],[3,4,5],[6,7,8],
        [0,3,6],[1,4,7],[2,5,8],
        [0,4,8],[2,4,6]
    ]
    for w in wins:
        if all(board[i] == player for i in w):
            return True
    return False
```

Check draw

```
def is_draw():
    return ' ' not in board
```

Minimax algorithm

```
def minimax(is_max):
    if check_winner('O'):
        return 1
    if check_winner('X'):
```

```
    return -1
if is_draw():
    return 0

if is_max:
    best = -math.inf
    for i in range(9):
        if board[i] == ' ':
            board[i] = 'O'
            best = max(best, minimax(False))
            board[i] = ' '
    return best
else:
    best = math.inf
    for i in range(9):
        if board[i] == ' ':
            board[i] = 'X'
            best = min(best, minimax(True))
            board[i] = ' '
    return best

# Computer move
def computer_move():
    best_score = -math.inf
    move = -1

    for i in range(9):
        if board[i] == ' ':
```

```
    board[i] = 'O'
    score = minimax(False)
    board[i] = ' '

    if score > best_score:
        best_score = score
        move = i

board[move] = 'O'

# Game loop
while True:
    print_board()

    pos = int(input("Enter position (1-9): ")) - 1

    if board[pos] != ' ':
        print("Invalid Move")
        continue

    board[pos] = 'X'

    if check_winner('X'):
        print_board()
        print("You Win!")
        break

    if is_draw():
```

```
print_board()
print("Draw!")
break
```

```
computer_move()
```

```
if check_winner('O'):
    print_board()
    print("Computer Wins!")
    break
```

```
if is_draw():
    print_board()
    print("Draw!")
    break
```

output:

```

  |  | 
--+---+--
  |  | 
--+---+--
  |  | 

Enter position (1-9): 6

  |  | O
--+---+--
  |  | X
--+---+--
  |  | 

Enter position (1-9): 2

  | X | O
--+---+--
O |  | X
--+---+--
  |  | 

Enter position (1-9): 4
Invalid Move

  | X | O
--+---+--
O |  | X
--+---+--
  |  | 

```

Result:

The Tic-Tac-Toe game was successfully implemented using the **Minimax algorithm**, allowing the computer to make optimal decisions and play intelligently.

Conclusion

The Python program successfully demonstrates the implementation of the **Minimax algorithm** for the Tic-Tac-Toe game. The AI evaluates all possible game states and selects the optimal move, ensuring the best possible outcome. This experiment illustrates the application of **game theory**, **adversarial search**, and **Artificial Intelligence** in developing intelligent game-playing agents.